

Shrubwise Data Challenge Report

Team: Dry shRub

Members: Uzair Gheewala, Sean Harmon
ugheewala@ucsd.edu, t2harmon@ucsd.edu

Question 1.

Our pipeline runs from data ingestion to modeling binary predictions which signal whether a shrub exists or does not exist at a specific location. At a high level, after intaking the data we run labeling and feature engineering pipelines to reframe the problem from unsupervised to supervised learning. From the modeling pipeline we proceed to the postprocessing pipeline and evaluation pipeline which are used to improve upon the original predictions and validate the model predictions based on a composite score.

We used data from all 6 sites: Calaveras Big Trees, DL Bliss, Independence Lake, Pacific Union College, Sedgwick, and Shaver Lake.

I'm going to summarize the main parts of the pipeline for brevity. At a high level our goal was to collect as much data as possible, create tilings to aggregate the data on and then model the tilings to predict if shrubs do or do not exist there. For ALS data we derived features from a canopy height model (e.g. distance to nearest canopy) and a KNN model (e.g. point cloud density). For 3DEP we used terrain information (e.g. slope, curvature) to derive new bands (e.g. Topographic Position Index, Hillshade). For RAP we created new bands using a uniform blur. For NAIP we used color channels to derive features like NDVI and VARI, as well as textures to generate bands (e.g. local binary patterns). Under the category of NAIP we performed a lot of general 2d image processing for new features (e.g. wavelets, filtering, gradients). The idea was that we could generate substantially more features by converting data to 2d and then applying these general operations. See our flow chart for the exact list of features used per pipeline (Feature Engineering subsections). These sources made up our features. When considering multiple sites, the only difference is that we added a site label to account for those differences, but otherwise all sites' features were ingested the same way if they had supporting data available.

We reserved TLS data for labeling as the resolution was higher which made it more precise in identifying shrubs. By removing TLS data from our main models we can have significantly faster inference time as the labels can be computed once. This labeling process involved using the provided R scripts for the shrub coordinates and radius. Using this information we rasterized a shrub by assuming circular shape for consistent interpretation regardless of reference frame. We then output confidence scores (ranging from 0 to 1) where pixels closer to the centroid were more likely to be a real shrub. For the confidence scores we left the option for local and global confidence scores as locally all the scores might be high or low, but globally they differ. Depending on the context one of these scoring schemas may be more appropriate.

For modeling we use a logistic regression and simple CNN as baselines. The primary model we used was UNet which has seen success in tumor detection. We found this shrub task to have a

similar framing as we don't actually know where the shrubs are, just like a scan might not actually know whether cancer is present or if it is a malignant tumor, without a person to follow up on the suspicion. We trained the model using a leave one out split on the sites (i.e. train 5 sites, predict the 6th). The idea was that the model should be site agnostic.

We wanted to use postprocessing as a second tier predictive model. The idea is that we could be highly confident in what is or is not a shrub after the first stage of modeling, at which point we could begin to tackle a wider range of predictive problems (e.g. shrub genus, shrub shape, 3d shrub predictions) on a per shrub level, as opposed to a per chunk level where multiple shrubs may coexist. In its current form we ended up using this stage for prediction tuning by estimating the best confidence threshold (e.g. $p > 0.5$, $p > 0.6$, etc.) to be confident in a shrub existing. We also enforced a minimum shrub size by considering all islands in the final prediction as a single shrub.

Evaluation is where we came back to the original unsupervised problem. When we came up with labels for modeling we entered the supervised domain, but to measure the original question of whether a shrub exists we couldn't hold on to some of these assumptions. We evaluated the shrub predictions made at each site by using a combination of Raster, Object, Multi-res and Site Summary scores. The Raster score took the average of dice similarity and Jaccard similarity for the site. The Object score took the average of a site's F1 score and its actual vs predicted count difference. The Multi-res score found the difference between the jaccard similarity at 1m and from multiple resolution decreases (2m, 5m and 10m). This ensured model consistency regardless of resolution. The Site Summary score averaged the expected and predicted differences for shrub cover as a percent of the site's area (counting per pixel of shrub), shrub count as a percent of the site's area (counting per shrub island), average shrub size in pixels, and median shrub size in pixels. Taking this holistic approach we weighted the resulting scores for a composite score equal to:

$$0.35 * \text{Raster} + 0.30 * \text{Object} + 0.15 * \text{Multires} + 0.20 * \text{Site Summary}$$

A score of 1 indicates an absolutely perfect prediction, while a score of 0 means dead wrong.

As mentioned at the start, this was a heavily shortened explanation as there was a lot of infrastructure regarding caching, chunking and other operations in between and during these stages to efficiently deal with the large amount of data available.

For preprocessing we calculated statistics on non-null values and tried to prevent null values (e.g. dividing by value plus epsilon). If a value was null we either filled it with a small number (e.g. -9999 for the canopy height model masks), a large number (e.g. 999999999 for missing dates) or otherwise, and most commonly, 0. We did not attempt any sophisticated interpolation as a value of 0 often indicates no signal, while any other value may indicate a signal in our model. To improve accuracy we only care if we are certain there is a signal. We created a config to better search the feature space. This included using a prespecified pixel/voxel side length corresponding to 1m, 2m, 5m or 10m depending on the run. In general the data started off fairly

clean as the values required dimension-appropriate coordinates to be usable and there were no ambiguous text features.

For flowcharting we used a [Lucid Chart](#).

Question 2.

On the full dataset (all sites) our logistic regression model had an overall accuracy of 25% on validation sets. It can cover some true positives but it's mostly picking up shrub context-ish signals. This is in the neighborhood of what we expected since log reg assumes a consistent reference frame for its weights. We were unfortunately not able to run the CNNs on the entire dataset from all sites due to time constraints so our notebook When we were testing with synthetic data that mimicked the white circle binary masks we would predict the simple CNN and UNet were able to both get around 98% overall accuracy so we expect similar results on the real data. When running our notebook we provide a small chunk of data from each site for predictions to ensure the inference time is more reasonable.

Question 3.

We thought the most important part of understanding this problem was understanding how people originally learned. People can go out into the world and say that something is a shrub (or whatever object) with a fairly high level of confidence even though at one point in the past no one ever had a uniform name for this class of objects. People have made language to force a supervised learning process upon themselves because that is a much more efficient method of learning. We thought this was exactly the problem listed here with shrubs. By mimicking this idea we were able to go beyond unsupervised algorithms which can be a limiting upper bound. We found that using TLS for labeling was ideal because it had higher precision that could help validate our confidence scores. If we were to look at TLS in light of our model we might be able to see a prediction representing the proportion of a shrub in that tile. Without TLS's precision it may be less clear if the model has a reason to show a lack of confidence since the gap between points is bigger. The biggest limitation was the lack of hard labels. We had to rely on soft labels and that can be dangerous. As soon as a soft label is wrong, it doesn't matter how well the rest of your process goes, you just can't make good predictions. You can't know which situation you are in without going out into the world yourself and collecting hard labels. Aside from this obvious limitation, we found the 3D data difficult to work with more generally. The discrete nature of data (i.e. lack of a continuous surface) and sparsity led to inconsistent groupings depending on how the chunk was sliced and 3d objects have slower run times compared to their same algorithms in 2d.

Question 4.

The main improvement to the pipeline would be with post-processing algorithms. We tried to exhaust meaningful feature engineering with as many algorithms as possible. During feature engineering we thought of as many features as we could so if we missed something it's because we didn't think about it, which was our original goal. We would want to expand post processing to have further sub tiers where the first sub tier might predict height, width, and other

shrub-specific characteristics. The second tier might predict genus or something that is determined by those composite features.

Question 5.

It would be most useful to have a better dictation of what the shrub looks like. Is it a 3d oval, a 3d rectangle? The 3d data feels sparse. It would be interesting if instead of discrete data being provided there was a continuous function that retains information on an individual shrub object's shape that could be resampled at the user's preferred rate. This could speed up some clustering algorithms and make object segmentation more consistent. With discrete data you can't necessarily distinguish leaves or other objects that make up a shrub because you don't know where the object is connected to the rest of the leaf. If there was a way to make a digital mold of a leaf and use that as input instead of discrete sampling, that might be helpful, for instance.